Assignment 4: Decision Trees and other classifiers on real dataset

This material, no matter whether in printed or electronic form, may be used for personal and non-commercial educational use only. Any reproduction of this material, no matter whether as a whole or in parts, no matter whether in printed or in electronic form, requires explicit prior acceptance of the authors.

Automatic Testing Guidelines

Automatic unittesting requires you to submit a notebook which contains strictly defined objects. Strictness of definition consists of unified shapes, dtypes, variable names and more.

Within the notebook, we provide detailed instruction which you should follow in order to maximise your final grade.

Name your notebook properly, follow the pattern in the template name:

Assignment_N_NameSurname_matrnumber

1. N - number of assignment
2. NameSurname - your full name where every part of the name starts with a capital letter, no spaces
3. matrnumber - you student number on ID card (with k, potentially with a leading zero)

Don't add any cells but use the ones provided by us. All cells have a unique ID so that the unit test can find it, so please do not add or remove any cell!

Always make sure that implemented functions have the correct output and given variables contain the correct data type. In the descriptions for every function you can find information on what datatype an output should have and you should stick to that in order to minimize conflicts with the unittest. Don't import any other packages than listed in the cell with the "imports" tag.

Questions are usually multiple choice (except the task description says otherwise) and can be answered by changing the given variables to either "True" or "False". "None" is counted as a wrong answer in any case!

Note: Never use variables you defined in another cell in your functions directly; always pass them to the function as a parameter. In the unitest, they won't be available either. If you want to make sure that everything is executable for the unittest, try executing cells/functions individually (instead of running the whole notebook).

Dataset

For this notebook to compile without problems, make sure that the required dataset files are stored in a folder called `dataset`! Also make sure that this folder and the additional Python file, provided via Moodle, are in the same folder as the notebook!

Task 1: Gini Impurity

In this task, we will recall the most important concepts of decision trees by walking you through a simple example. On the way you have to solve some exercises to gain basic insights. Let's start with a simple toy dataset for one tree and give a defintion of the concepts needed for the following tasks.

- **Code 1.1**: Implement the necessary calculations in the function `calc_gini` and return the 4 solutions (**Note:** Your implementation should work for any dataset similar to the toy dataset (i.e. binary labels, two dimensional)):
  1. Calculate the Gini impurity for our toy dataset.
  2. Calculate the Gini impurity for the top/right leaf in a given x_2- or x_1-split.
  3. Calculate the Gini impurity for the bottom/left leaf in a given x_2- or x_1-split.
  4. Calculate the Gini impurity gain for a given split in the toy dataset.
- **Question 1.1**:
  - Answer some questions about the results of **Code 1.1**.

In [1]:
```
# Nothing to do here, just run the cell.
import time
import numpy as np

import sklearn
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix

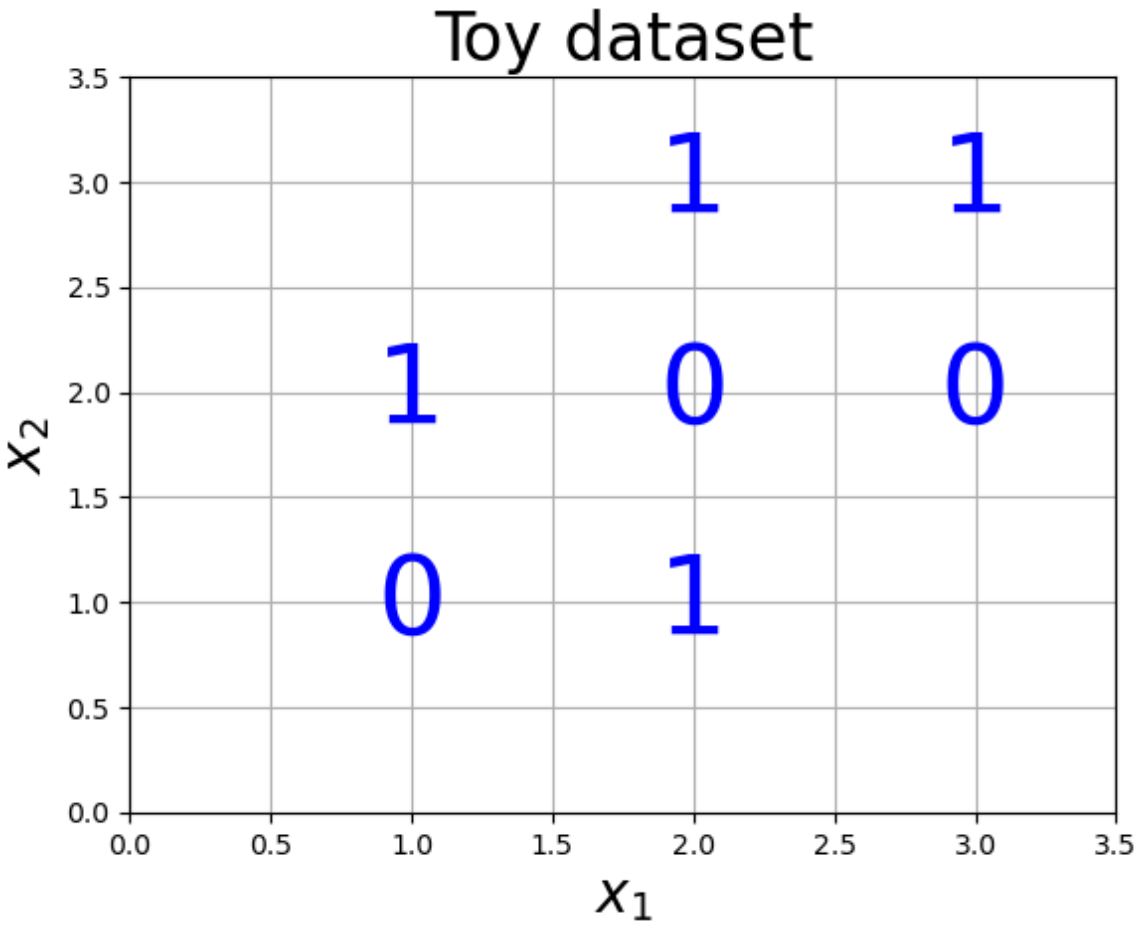
from mnist_loader import MNIST
import matplotlib.pyplot as plt
# Set random seed to ensure reproducible runs
RSEED = 50
```

In [2]:
```
# Nothing to do here, just run the cell.
X_toy = np.array(
    [[2, 2],
     [2, 1],
     [2, 3],
     [1, 2],
     [1, 1],
     [3, 3],
     [3, 2]]
)

y_toy = np.array([0, 1, 1, 1, 0, 1, 0])
```

In [3]:
```
# Nothing to do here, just run the cell.
# Plot each point as the label.
for x1, x2, label in zip(X_toy[:, 0], X_toy[:, 1], y_toy):
    plt.text(x1, x2, str(label), fontsize = 40, color = 'b',
             ha='center', va='center')

plt.grid()
plt.xlim((0, 3.5))
plt.ylim((0, 3.5))
plt.xlabel('$x_1$', size = 20); plt.ylabel('$x_2$', size = 20)
plt.title('Toy dataset', size = 24)
plt.show()
```



A **Decision Tree Classifier (DTC)** builds a decision tree based on the features of the data. This is equivalent to subdividing the feature space. Let's consider the example above and apply a simple heuristics. In the first step we try to subdivide the space such that we obtain the *largest possible leaf (subdivision)* that contains only **one class**.

We first look at the feature x_2, i.e. a horizontal division of the space. For example, we could divide the space at the specific threshold $x_2 = 2.5$. Then we end up having a group of samples with features $x_2 > 2.5$ and homogenous class label 1, i.e. the two points with coordinates $\{(2, 3), (3, 3)\}$. If we instead look at the feature x_1, corresponding to a vertical division of the space, we cannot find an equally large or larger group of samples with the same label. Therefore, our first node in the tree is: $x_2 \leq 2.5$, i.e. we split the space and repeat the same procedure on each of the two

leafs. In our case we are done with the top leaf (since both samples have the same class) and only need to repeat the procedure on the bottom leaf.

Our simple heuristics from above fails on the bottom node. We need a better criterion to decide which splits to make.
Nowadays the most frequently used one is called the **Gini Impurity**.

The Gini Impurity is a measure of how often a randomly chosen element from the set would be incorrectly labeled, if it was randomly labeled according to the distribution of labels in the subset.

What does that mean?

1. Let us suppose we have 2 labels and let p_1, p_2 be the fractions of points labeled with labels 1 and 2 (note: $p_1 + p_2 = 1$) .
2. The probabilty to choose a point with label 1 is p_1.
3. The probability to choose label 2 is $p_2 = 1 - p_1$.
4. Therefore, the probability to label a point of label 1 with label 2 is $p_1 \cdot p_2 = p_1 \cdot (1 - p_1) = p_1 - p_1^2$.
5. Analogously, the probability for points with label 2 to be labeled with 1 is $p_2 \cdot (1 - p_2) = p_2 - p_2^2$.
6. The Gini Impurity is the sum over both: $p_1 - p_1^2 + p_2 - p_2^2 = p_1 + p_2 - p_1^2 - p_2^2 = 1 - p_1^2 - p_2^2$

The above reasoning is easy to generalize to the case where the number of labels M is larger than two: $M > 2$.
The formula for the given dataset Z is simply $I_G(Z) = 1 - \sum_{k=1}^{M} p_k(Z)^2$, where $p_k(Z)$ is the frequency of points with labels k in the dataset Z.

The Gini Impurity Gain is the amount of "impurity" we get rid of for a specific split s.

Let's assume that we get the partition $Z_{s,1}, \ldots, Z_{s,K_s}$ of Z after applying s.

Then the impurity gain is $g_G(Z, s) = I_G(Z) - \sum_{t=1}^{K_s} \frac{|Z_{s,t}|}{|Z|} \cdot I_G(Z_{s,t})$

Code 1.1 (15 Points):

In [4]:
```
def calc_gini(X: np.ndarray, y: np.ndarray, split: float, axis: int) -> t
    """Function that calculates the Gini Impurity of the whole dataset an
    Also returns the impurity gain from this specific split.

    Parameters
    ----------
    X : (N, 2) np.ndarray
        Data matrix.
    y : (N,) np.ndarray
        Binary labels.
    split : float
        Value at which the split is performed
    axis : int
        Axis on which the split is performed, 0 for the first feature (=x
```

```
    Returns
    -------
    tuple of float
        - gini_impurtiy : float
            The gini impurity of the whole dataset.
        - gini_impurity_top / gini_impurity_right : float
            The gini impurity for the top or right split depending on the
        - gini_impurity_bottom / gini_impurity_left : float
            The gini impurity for the bottom or left split depending on t
        - impurity_gain : float
            The impurity gain of the performed split.
    """

    # YOUR CODE HERE
    # I_G(Z) = 1 - (p_1^2 + p_2^2)
    # I_G(Z) = 1 - (p_1^2 + (1 - p_1)^2)
    gini_impurity = 1 - (np.mean(y)**2 + (1 - np.mean(y))**2)

    if axis == 0:
        top = y[X[:, axis] > split]
        bottom = y[X[:, axis] <= split]
    else:
        top = y[X[:, axis] > split]
        bottom = y[X[:, axis] <= split]

    gini_impurity_top = 1 - (np.mean(top)**2 + (1 - np.mean(top))**2)
    gini_impurity_bottom = 1 - (np.mean(bottom)**2 + (1 - np.mean(bottom)
    #print(len(top), len(bottom))
    impurity_gain = gini_impurity - ((len(top)/len(y) * gini_impurity_top

    return gini_impurity, gini_impurity_top, gini_impurity_bottom, impuri
```

In [5]:
```
# DO NOT DELETE THIS CELL!
x1_split_point= 1.5
gini_impurity, gini_right, gini_left, impurity_gain_0 = calc_gini(X_toy,

x2_split_point= 2.5
_, gini_top, gini_bottom, impurity_gain_1 = calc_gini(X_toy, y_toy, x2_sp

assert isinstance(gini_impurity, float), "The gini impurity is not a floa
assert np.isclose(gini_impurity, 0.4898, atol=1e-4), "The gini impurity i
```

In [6]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(gini_right, float), "The gini impurity for the right sp
assert np.isclose(gini_right, 0.4800, atol=1e-4), "The gini impurity for
assert np.isclose(gini_top, 0, atol=1e-4), "The gini impurity for the top
```

In [7]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(gini_left, float), "The gini impurity for the left spli
assert np.isclose(gini_left, 0.5, atol=1e-4), "The gini impurity for the
assert np.isclose(gini_bottom, 0.48, atol=1e-4), "The gini impurity for t
```

In [8]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(impurity_gain_0, float), "The gini impurity gain is not
assert np.isclose(impurity_gain_0, 0.0041, atol=1e-4), "The gini impurity
assert np.isclose(impurity_gain_1, 0.1469, atol=1e-4), "The gini impurity
```

In [9]:
```
# Nothing to do here, just run the cell.
print(f"Results for x1 = {x1_split_point}:\n")
print(
    f"Gini impurity for the entire dataset: {gini_impurity:0.4f}\n"
    f"Gini impurity right leaf: {gini_right:0.4f}\n"
    f"Gini impurity left leaf: {gini_left:0.4f}\n"
    f"Gini impurity gain: {impurity_gain_0:0.4f}\n\n"
)

print(f"Results for x2 = {x2_split_point}:\n")
print(
    f"Gini impurity for the entire dataset: {gini_impurity:0.4f}\n"
    f"Gini impurity top leaf: {gini_top:0.4f}\n"
    f"Gini impurity bottom leaf: {gini_bottom:0.4f}\n"
    f"Gini impurity gain: {impurity_gain_1:0.4f}"
)
```

```
Results for x1 = 1.5:

Gini impurity for the entire dataset: 0.4898
Gini impurity right leaf: 0.4800
Gini impurity left leaf: 0.5000
Gini impurity gain: 0.0041


Results for x2 = 2.5:

Gini impurity for the entire dataset: 0.4898
Gini impurity top leaf: 0.0000
Gini impurity bottom leaf: 0.4800
Gini impurity gain: 0.1469
```

Question 1.1 (3 Points):

Based on the results of Gini impurity for the two different split locations of x_2, which of the following statements are correct?

a1_) The right leaf of the split at $x_1 = 1.5$ has smaller Gini impurity than the top leaf of the split at $x_2 = 2.5$
b1_) The Gini impurity of the entire dataset does not depend on the split location.
c1_) The Gini impurity gain of the split at $x_1 = 1.5$ is smaller than at $x_2 = 2.5$, indicating that $x_1 = 1.5$ is a worse splitting point.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a1_)** is correct, define a variable `a1_` and set it to `True`, the same applies to **b1_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [10]:
```
# YOUR CODE HERE
a1_ = False
b1_ = True
c1_ = True
```

In [11]:
```
# DO NOT DELETE THIS CELL!
assert a1_ is not None, "Store True/False!"
assert a1_ in [True, False], "Invalid Answer!"
```

In [12]:
```
# DO NOT DELETE THIS CELL!
assert b1_ is not None, "Store True/False!"
assert b1_ in [True, False], "Invalid Answer!"
```

In [13]:
```
# DO NOT DELETE THIS CELL!
assert c1_ is not None, "Store True/False!"
assert c1_ in [True, False], "Invalid Answer!"
```

Task 2: Train a simple decision tree

Next, you should provide a Python routine for the previous example.

- **Code 2.1**: In the cells below is the function `dec_tree` where you have to implement the following:
  - Train a decision tree (have a look at DecisionTreeClassifier) on the given dataset X_{train}, y_{train} and remember to pass the random seed `seed` defined as function parameter.
  - Return the fitted DecisionTreeClassifier, the number of tree nodes, the maximum depth of the tree, and the accuracy of the tree on the given X_{test}, y_{test}.

Code 2.1 (12 Points):

In [14]:
```
def dec_tree(X_train: np.ndarray, y_train: np.ndarray, X_test: np.ndarray
    """Trains a decision tree and returns certain attributes of the recei

    Important: Use sklearn's DecisionTreeClassifier for this task, don't

    Parameters
    ----------
    X_train : (N, K) np.ndarray
        Data matrix used for fitting the tree.
    y_train : (N,) np.ndarray
        Binary labels of the train data.
    X_test : (N, K) np.ndarray
        Data matrix used for computing the accuracy of the tree.
    y_test : (N,) np.ndarray
        Binary labels of the test data.
    seed : int
        Seed for reproducibility.

    Returns
    -------
    tuple of DecisionTreeClassifier, int, float
        - tree : DecisionTreeClassifier
            The fitted tree.
        - nr_nodes : int
            The number of nodes in the fitted decision tree.
        - max_depth : int
```

```
            The max depth of the tree.
        - accuracy : float
            The achieved accuracy on the given dataset.
    """
    # YOUR CODE HERE
    dec_tree = DecisionTreeClassifier(random_state=seed)
    dec_tree.fit(X_train, y_train)
    nr_nodes = dec_tree.tree_.node_count
    max_depth = dec_tree.tree_.max_depth
    accuracy = accuracy_score(y_test, dec_tree.predict(X_test))
    tree = dec_tree
    return tree, nr_nodes, max_depth, accuracy
```

In [15]:
```
# DO NOT DELETE THIS CELL!
fitted_tree, nr_nodes, max_depth, acc = dec_tree(X_toy, y_toy, X_toy, y_t
assert isinstance(fitted_tree, DecisionTreeClassifier), "The returned tre
assert isinstance(nr_nodes, int), "The number of nodes is not an integer!
assert nr_nodes == 11, "The number of nodes is wrong!"
```

In [16]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(max_depth, int), "The max depth is not an integer!"
assert max_depth == 4, "The max depth is wrong!"
```

In [17]:
```
# DO NOT DELETE THIS CELL!
assert isinstance(acc, float), "The accuracy is not a float!"
assert acc == 1, "The accuracy is wrong!"
```

In [18]:
```
# Nothing to do here, just run the cell.
print(f'Decision tree has {nr_nodes} nodes with maximum depth {max_depth}
print(f'Model accuracy: {acc}')
```

```
Decision tree has 11 nodes with maximum depth 4.
Model accuracy: 1.0
```

Task 3: Decision tree on a real data set

Now we will apply the classifier to a well known real-world benchmark data set, namely the "Fashion MNIST" dataset. It consists of images of clothing, like sneakers and shirts. It was created to be an alternative to the famous MNIST benchmark dataset, which is nowadays considered as too easy for the most recent algorithms. Let us first load the train and test set, using the files provided on Moodle. The train and test datasets are represented as flattened pixel arrays ($28 \times 28 = 784$ pixels), and the label vector indicates the different classes (0 to 9). The following cells train and evaluate a simple DecisionTreeClassifier on a small subset of the Fashion MNIST dataset, to give you a baseline.

- **Code 3.1**: In order to obtain a better performance, we apply a hyperparameter search.
  - To this end implement a factory function called `create_param_grid` that simply creates a parameter grid (dictionary) which contains the following quantities:
    - `criterion` : 'gini' and 'entropy'
    - `max_depth` : 10, 50 and 100

    - `splitter`: 'random' and 'best'
    - **Hint:** Have a look at the documentation of RandomizedSearchCV to get an idea on how this parameter grid should look like.
- Implement the function `train_dec_tree` that applies RandomizedSearchCV to train a DecisionTreeClassifier using Cross-Validation and returns the best fitted model on the train data and its `best_params_` dictionary (have a look at the documentation).
- Again, don't forget to pass the seed in **(both!)** the **DecisionTreeClassifier** and **RandomizedSearchCV**.

In [19]:
```
# Nothing to do here, just run the cell.
data = MNIST('./dataset/')
img_train, labels_train = data.load_training()
n_train = 12000
n_test = 2000
X_train = np.array(img_train)[:n_train]
y_train = np.array(labels_train)[:n_train]
print(X_train.shape, y_train.shape)

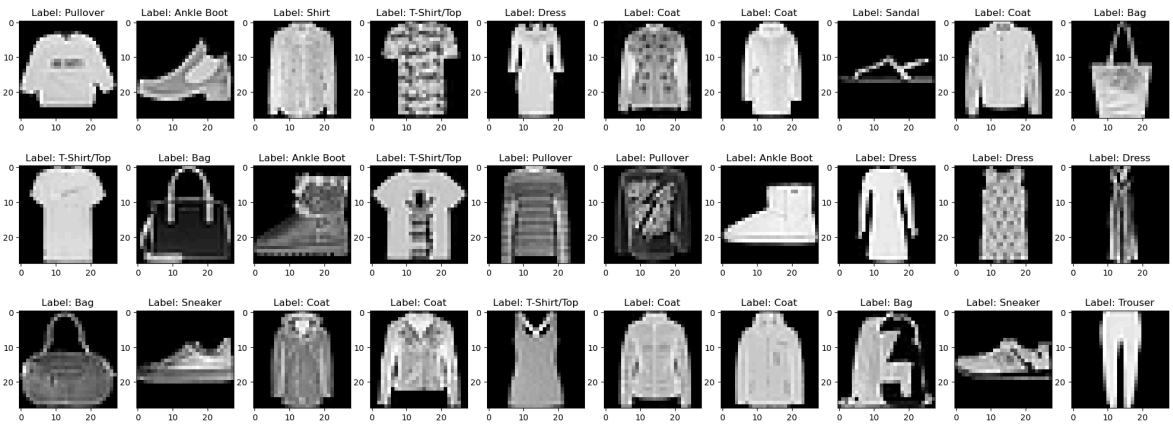
img_test, labels_test = data.load_testing()
X_test = np.array(img_test)[:n_test]
y_test = np.array(labels_test)[:n_test]
print(X_train.shape, y_test.shape)
```

```
(12000, 784) (12000,)
(12000, 784) (2000,)
(12000, 784) (2000,)
```

To know what we are dealing with, let us plot some of the images:

In [20]:
```
# Nothing to do here, just run the cell.
labels = ['T-Shirt/Top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal'
plt.figure(figsize=(25, 9))
for i in range(30):
    plt.subplot(3, 10, i + 1)
    two_d = (np.reshape(X_train[i], (28, 28))).astype(np.uint8)
    plt.imshow(two_d, interpolation='nearest', cmap='gray')
    plt.title('Label: {0}'.format(labels[y_train[i]]))

plt.show()
```



The function `get_evaluation` should additionally help you to compute accuracies and provide confusion matrices and appropriate heatmaps.

In [21]:
```
# Nothing to do here, just run the cell.
def get_evaluation(model: sklearn.base.BaseEstimator, X_test: np.ndarray,
    """Evaluates the given model and plots a confusion matrix. Also measu

    Parameters
    ----------
    model : sklearn.base.BaseEstimator
        Trained model that implements the `predict` method.
    X_test : (N, K) np.ndarray
        Data matrix for testing.
    y_test : (N,) np.ndarray
        True labels for the test data.

    Returns
    -------
    tuple of float
        - accuracy : float
            The accuracy of the fitted classifier on the given test set.
        - inference_time : float
            The time it took to compute the predictions.
    """
    start = time.time()
    y_pred = model.predict(X_test)
    inference_time = time.time()-start
    accuracy = accuracy_score(y_test, y_pred)
    conf_mat = confusion_matrix(y_test, y_pred)

    print('\nAccuracy of classifier on test image data: ', accuracy)
    print('\nConfusion matrix: \n', conf_mat)
    print('\nTime: ', inference_time)
    print()

    plt.matshow(conf_mat)
    plt.title('Confusion matrix')
    plt.colorbar()
    plt.ylabel('True label')
    plt.xlabel('Predicted label')
    plt.show()

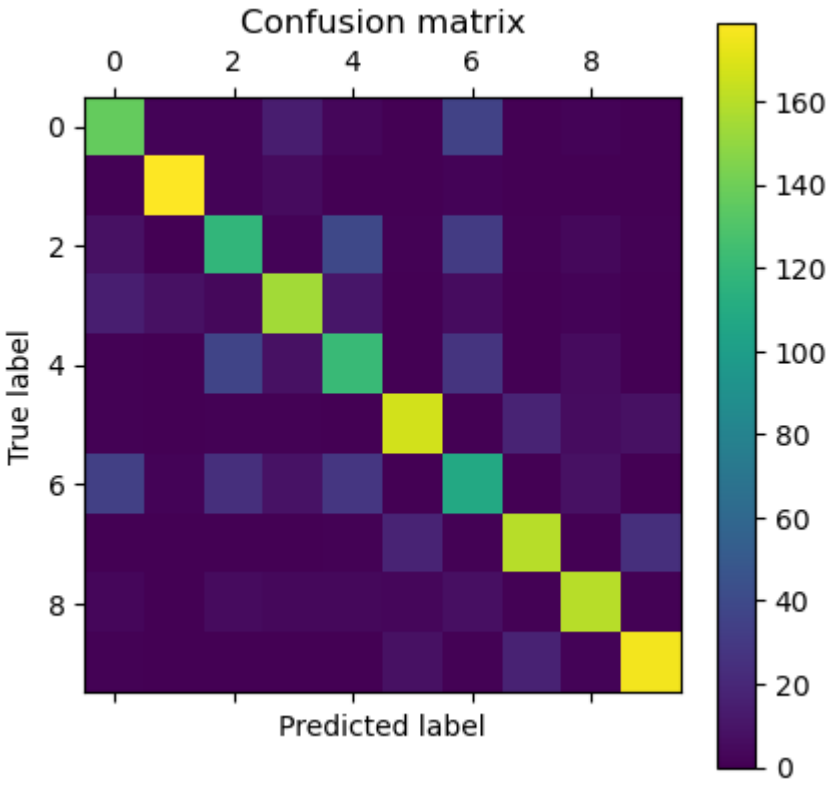
    return accuracy, inference_time
```

In [22]:
```
# Nothing to do here, just run the cell.
model, _, _, _ = dec_tree(X_train, y_train, X_test, y_test, RSEED)
acc_fixed, time_fixed = get_evaluation(model, X_test, y_test)
```

```
Accuracy of classifier on test image data:  0.74

Confusion matrix:
 [[137   2   2  14   3   0  35   0   2   0]
 [  1 179   2   5   0   0   2   0   0   0]
 [  8   0 118   2  39   1  31   1   4   1]
 [ 15   8   4 154  11   0   6   0   2   0]
 [  1   0  37   8 121   0  27   0   5   0]
 [  1   0   1   1   0 167   0  18   6   8]
 [ 34   2  24   9  28   0 108   0   8   0]
 [  0   0   0   0   1  18   1 160   0  24]
 [  3   0   5   4   4   3   7   1 160   1]
 [  1   0   0   0   0   8   1  17   2 176]]

Time:  0.0019135475158691406
```



Code 3.1 (25 Points):

In [23]:
```python
def create_param_grid() -> dict:
    """Creates a parameter grid for hyperparameter tuning of a decision t

    Returns
    -------
    dict
        Dictionary containing parameter names (`criterion`, `max_depth`,
        and their respective values to be used in grid search.
        - 'criterion': list of str
            Specifies the function to measure the quality of a split. Opt
        - 'max_depth': list of int
            Specifies the maximum depth of the tree. Options are 10, 50,
        - 'splitter': list of str
            Specifies the strategy used to split at each node. Options ar
    """
    # YOUR CODE HERE
```

```
    param_grid = {'criterion': ['entropy', 'gini'],
                  'max_depth': [10, 50, 100],
                  'splitter': ['random', 'best']}

    return param_grid
```

In [24]:
```
# DO NOT DELETE THIS CELL!
param_grid = create_param_grid()
assert isinstance(param_grid, dict), "The parameter grid is not a diction
assert len(param_grid.keys()) == 3, "The number of keys is not correct!"
assert set(param_grid["criterion"]) == {"entropy", "gini"}, "The criterio
assert set(param_grid["max_depth"]) == {10, 50, 100}, "The max_depth does
assert set(param_grid["splitter"]) == {"random", "best"}, "The splitter d
```

In [25]:
```
def train_dec_tree(X_train: np.ndarray, y_train: np.ndarray, param_grid:
    """Trains a decision tree using cross-validation and returns the best

    Important: Again use (only!) the implementations from sklearn already

    Parameters
    ----------
    X_train : (N, K) np.ndarray
        Data matrix used for fitting the tree.
    y_train : (N,) np.ndarray
        Labels of the train data.
    param_grid : dict
        Dictionary of parameters for the grid search (RandomizedSearchCV)
    n_iter : int
        Number of iterations (RandomizedSearchCV)
    cv : int
        Number of folds in CV (RandomizedSearchCV)
    seed : int
        Seed for reproducibility, feed to both RandomizedSearchCV and the

    Returns
    -------
    tuple of RandomizedSearchCV, dict, float
        - model : RandomizedSearchCV
            Fitted best model.
        - best_params : dict
            Best model parameters as dict.
    """
    # YOUR CODE HERE
    tree = DecisionTreeClassifier(random_state=seed)
    model = RandomizedSearchCV(tree, param_grid, n_iter=n_iter, cv=cv, ra
    model.fit(X_train, y_train)
    best_params = model.best_params_
    return model, best_params
```

In [26]:
```
# DO NOT DELETE THIS CELL!
model_dec_tree, params_dec_tree = train_dec_tree(X_train, y_train, param_
assert isinstance(model_dec_tree, RandomizedSearchCV), "The best decision
assert isinstance(params_dec_tree, dict), "The best params should be a di
```

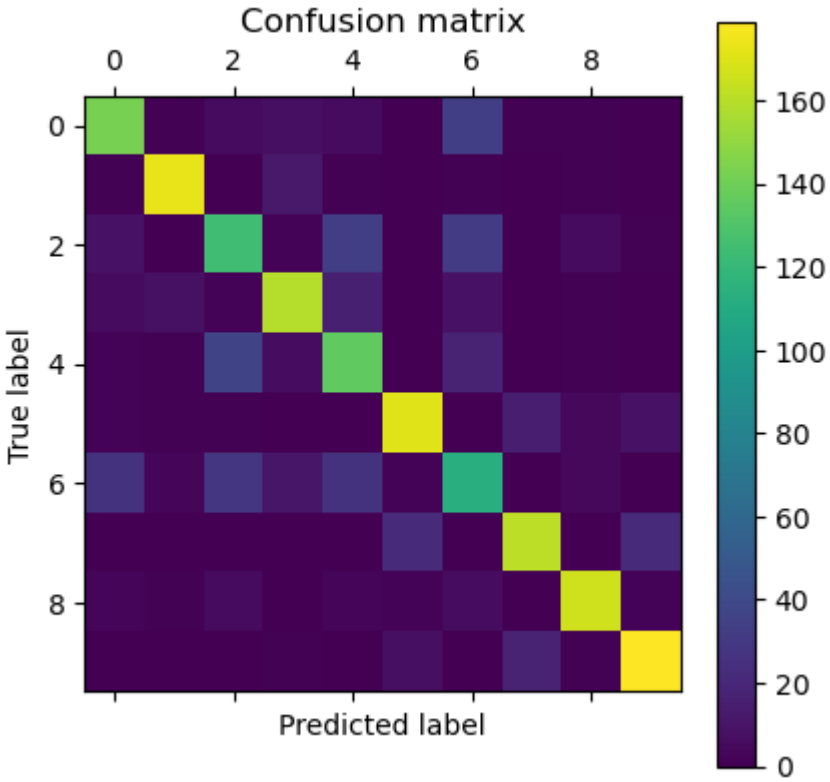
In [27]:
```
# Nothing to do here, just run the cell.
print("The best parameters are: {}".format(params_dec_tree))
acc_best, time_best = get_evaluation(model_dec_tree, X_test, y_test)
```

```
The best parameters are: {'splitter': 'best', 'max_depth': 50, 'criterio
n': 'entropy'}

Accuracy of classifier on test image data:  0.761

Confusion matrix:
 [[142   1   5   7   5   0  33   1   1   0]
 [  1 173   0  12   1   0   1   0   1   0]
 [  9   0 124   2  33   0  31   0   5   1]
 [  5   8   2 159  16   0   9   0   1   0]
 [  2   1  37   6 135   0  17   0   1   0]
 [  2   1   1   0   0 170   0  15   4   9]
 [ 26   3  28  11  26   2 113   0   4   0]
 [  0   0   0   0   0  22   0 161   0  21]
 [  3   1   5   0   3   2   6   0 166   2]
 [  0   0   0   1   0   7   0  17   1 179]]

Time:  0.00366973876953125
```



If you did the task correctly, you should obtain a slightly better accuracy than before.

Task 4: Comparison of different classifiers

In this task we compare the performace of the Decision Tree Classifier to two other classifiers, namely KNN and SVM, also evaluated on the Fashion MNIST dataset:

- **Code 4.1**:
  - Implement a function `train_kNN` that creates and fits a KNN classifier with the given `n_neighbors`, `weights`, and `p` parameters (have a look at KNeighborsClassifier). This function should return the fitted model.
- **Code 4.2**:

    - Implement a function `train_SVM` that creates and fits a SVM classifier with `C`, `kernel`, and `gamma` parameters (have a look at SVC). Similar to `train_kNN`, this function should again return the fitted model.
- **Plot 4.3**:
    - Now, plot the accuracies of the four classifiers (decision tree with fixed parameters, decision tree with the best parameters, KNN, and SVM) in a **bar plot** with y-axis in the range of $[0, 1]$ to compare them. To this end, implement the function `plot_accuracies`.
    - Make another **bar plot** with the four inference times (in seconds) of the four classifiers. Make sure to use the log-scale on the y-axis. Similar to the previous plot, implement the function `plot_times`.
    - Make sure to label all axis and give you plots a fitting title!
- **Question 4.3**:
    - Afterwards, answer some questions that correspond to your observations.

Code 4.1 (10 Points):

In [28]:
```
def train_KNN(X_train: np.ndarray, y_train: np.ndarray, n_neighbors: int,
    """Trains a KNN classifier on the given dataset and computes the accu

    Again use the sklearn implementation, but no need to set a seed for t

    Parameters
    ----------
    X_train : (N, K) np.ndarray
        Data matrix used for fitting the classifier.
    y_train : (N,) np.ndarray
        Labels of the train data.
    n_neighbors : int
        KNN parameter, number of neighbors.
    weights : str
        Knn parameter, mode for weights.
    p : float
        Power parameter for the Minkowski metric (see documentation: KNei

    Returns
    -------
    model : KNeighborsClassifier
            The fitted classifier.
    """
    # YOUR CODE HERE
    knn = KNeighborsClassifier(n_neighbors=n_neighbors, weights=weights,
    knn.fit(X_train, y_train)

    return knn
```

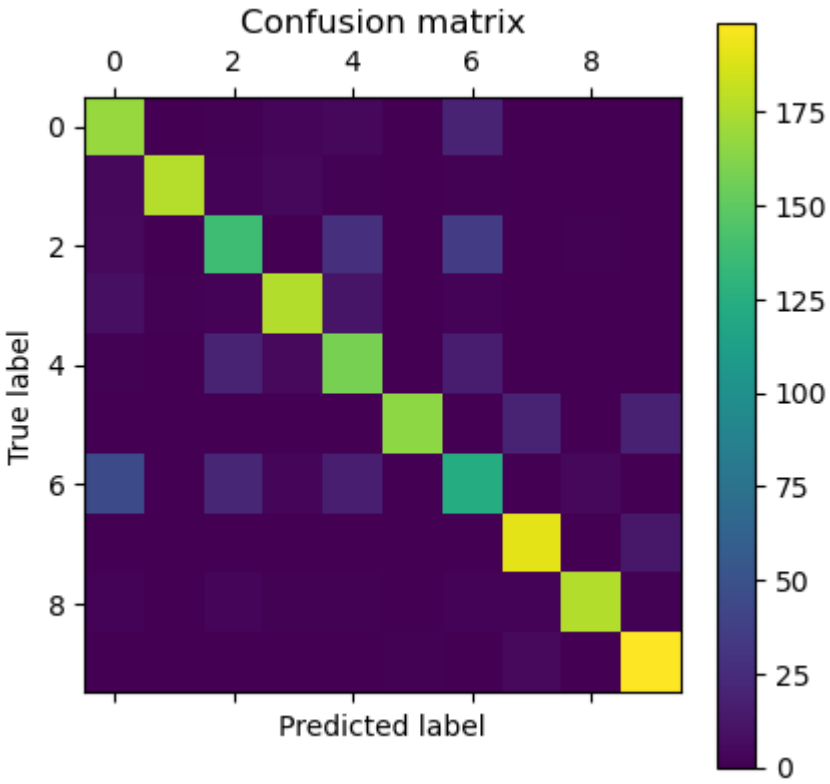
In [29]:
```
# DO NOT DELETE THIS CELL!
knn_model = train_KNN(X_train, y_train, 5, 'distance', 1)
assert isinstance(knn_model, KNeighborsClassifier), "The returned model i
assert knn_model.n_neighbors == 5, "The number of neighbors is wrong!"
assert knn_model.weights == "distance", "The 'weights' parameter is wrong
assert knn_model.p == 1, "The 'P' parameter is wrong!"
```

In [30]:
```
# Nothing to do here, just run the cell.
knn_acc, knn_time = get_evaluation(knn_model, X_test, y_test)
```

```
Accuracy of classifier on test image data:  0.835

Confusion matrix:
 [[168   0   1   3   4   0  19   0   0   0]
 [  4 177   2   4   1   0   1   0   0   0]
 [  5   0 137   0  27   0  35   0   1   0]
 [  8   1   2 176  11   0   2   0   0   0]
 [  1   0  20   5 158   0  15   0   0   0]
 [  0   0   0   0   0 165   0  19   0  18]
 [ 45   0  21   3  17   0 123   0   4   0]
 [  0   0   0   0   0   0   0 191   0  13]
 [  2   0   3   1   1   0   2   2 176   1]
 [  0   0   0   0   0   1   0   5   0 199]]

Time:  12.046984195709229
```



4.2 Code (10 Points):

In [31]:
```
def train_SVM(X_train: np.ndarray, y_train: np.ndarray, C: float, kernel:
    """ Trains an SVM classifier on the given dataset.

    Important: Again use the sklearn implementation provided, and don't f

    Parameters
    ----------
    X_train : (N, K) np.ndarray
        Data matrix used for fitting the classifier.
    y_train : (N,) np.ndarray
        Labels of the train data.
    C : float
        SVM parameter for regularization.
```

```
    kernel : str
        SVM parameter for the type of kernel being.
    gamma : str
        SVM parameter for kernel coefficient.
    seed : int
        Seed for reproducibility.

    Returns
    -------
    model : SVC
        The fitted classifier.
    """
    # YOUR CODE HERE
    svm = SVC(C=C, kernel=kernel, gamma=gamma, random_state=seed)
    svm.fit(X_train, y_train)
    return svm
```

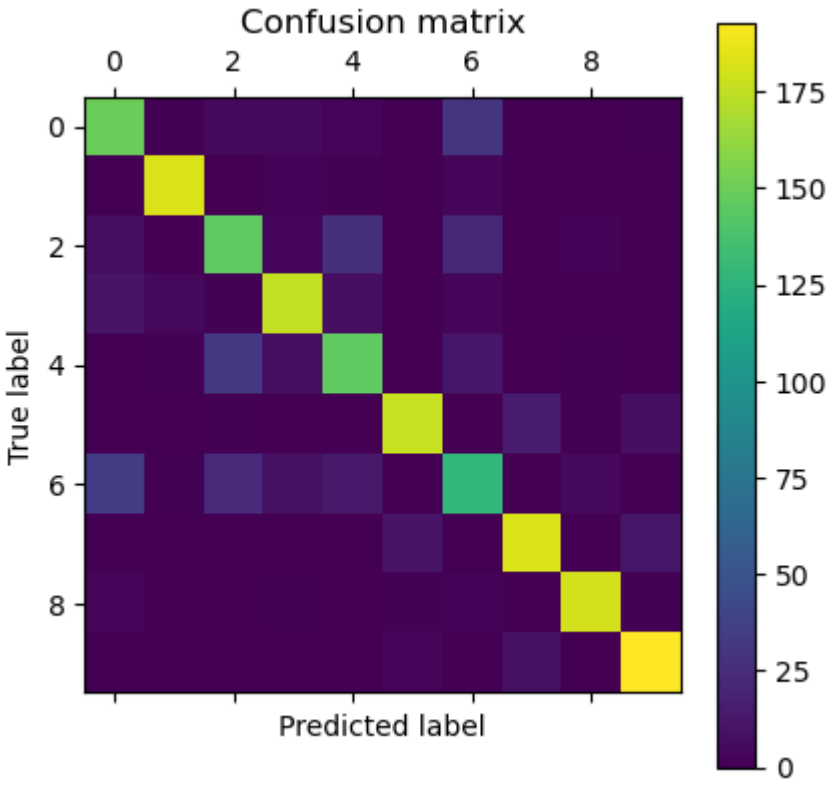
In [32]:
```
# DO NOT DELETE THIS CELL!
svm_model = train_SVM(X_train, y_train, 10, 'poly', 'auto', RSEED)
assert isinstance(svm_model, SVC), "The returned model is not a SVC!"
assert svm_model.C == 10, "The regularization parameter 'C' is wrong!"
assert svm_model.kernel == "poly", "The 'kernel' parameter is wrong!"
assert svm_model.gamma == "auto", "The 'gamma' parameter is wrong!"
```

In [33]:
```
# Nothing to do here, just run the cell.
svm_acc, svm_time = get_evaluation(svm_model, X_test, y_test)
```

```
Accuracy of classifier on test image data:  0.8295

Confusion matrix:
 [[150   1   5   5   3   0  30   0   0   1]
 [  0 183   0   2   1   0   3   0   0   0]
 [  7   0 145   3  26   0  22   0   2   0]
 [ 10   4   1 175   7   0   3   0   0   0]
 [  0   1  32   7 146   0  12   0   1   0]
 [  0   0   1   0   0 177   0  15   1   8]
 [ 34   1  24   9  13   0 128   0   4   0]
 [  0   0   0   0   0  10   0 182   0  12]
 [  3   0   0   1   0   1   2   0 180   1]
 [  0   0   0   0   0   3   0   9   0 193]]

Time:  2.0587244033813477
```



Plot 4.3 (7 Points):

In [34]:
```
def plot_accuracies(accuracies: list[float]):
    """Creates a bar-plot from the classifier accuracies with a range of

    Parameters
    ----------
    accuracies : list
        List of 4 accuracies from the previous exercises.
            - dec_tree_fixed : float
                Accuracy of the decision tree with fixed parameters.
            - dec_tree_bets : float
                Accuracy of the decision tree with best parameters.
            - kNN : float
                Accuracy of the kNN classifier.
            - SVM : float
                Accuracy of the SVM classifier.
    """
    # YOUR CODE HERE
    plt.bar(['D.T. (fixed)', 'D.T. (best)', 'kNN', 'SVM'], accuracies)
    plt.ylim(0, 1)
    plt.ylabel('Accuracy')
    plt.title('Acuracies of 4 Classifiers')
    plt.show()
```

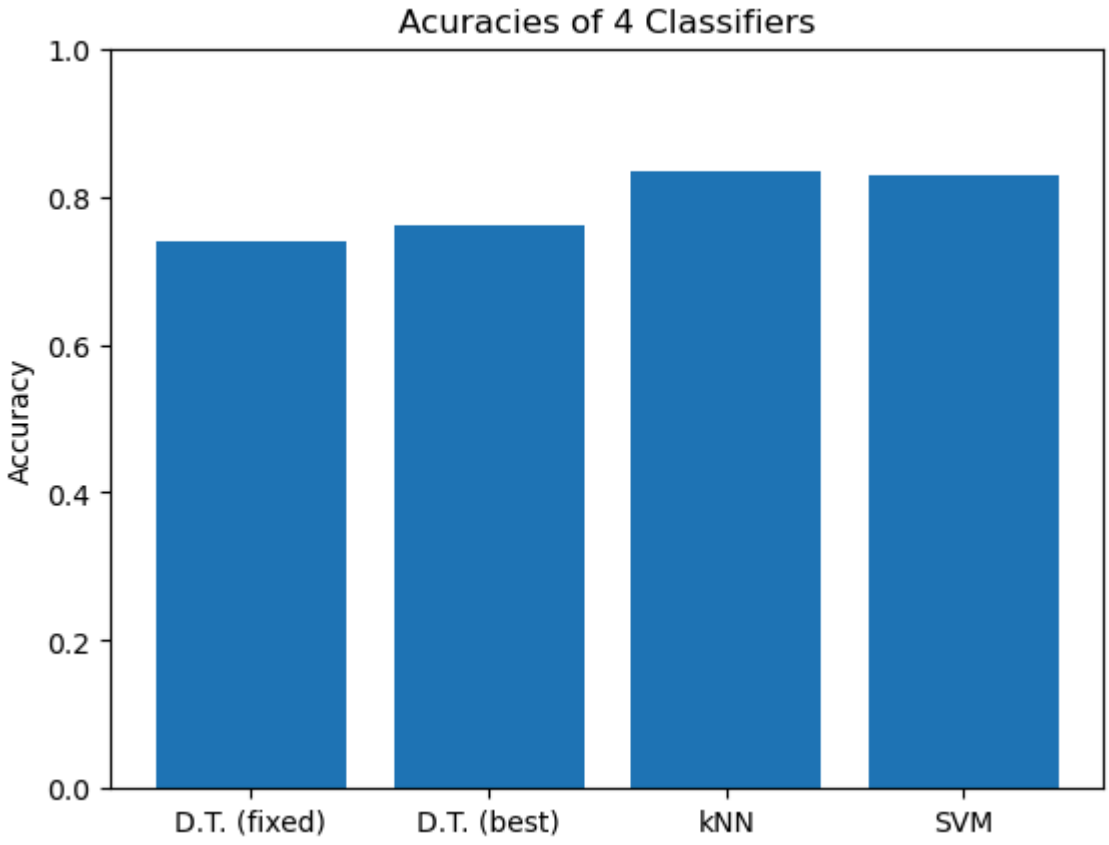
In [35]:
```
def plot_times(times: list[float]):
    """Creates a bar-plot from the classifier inference times using the l

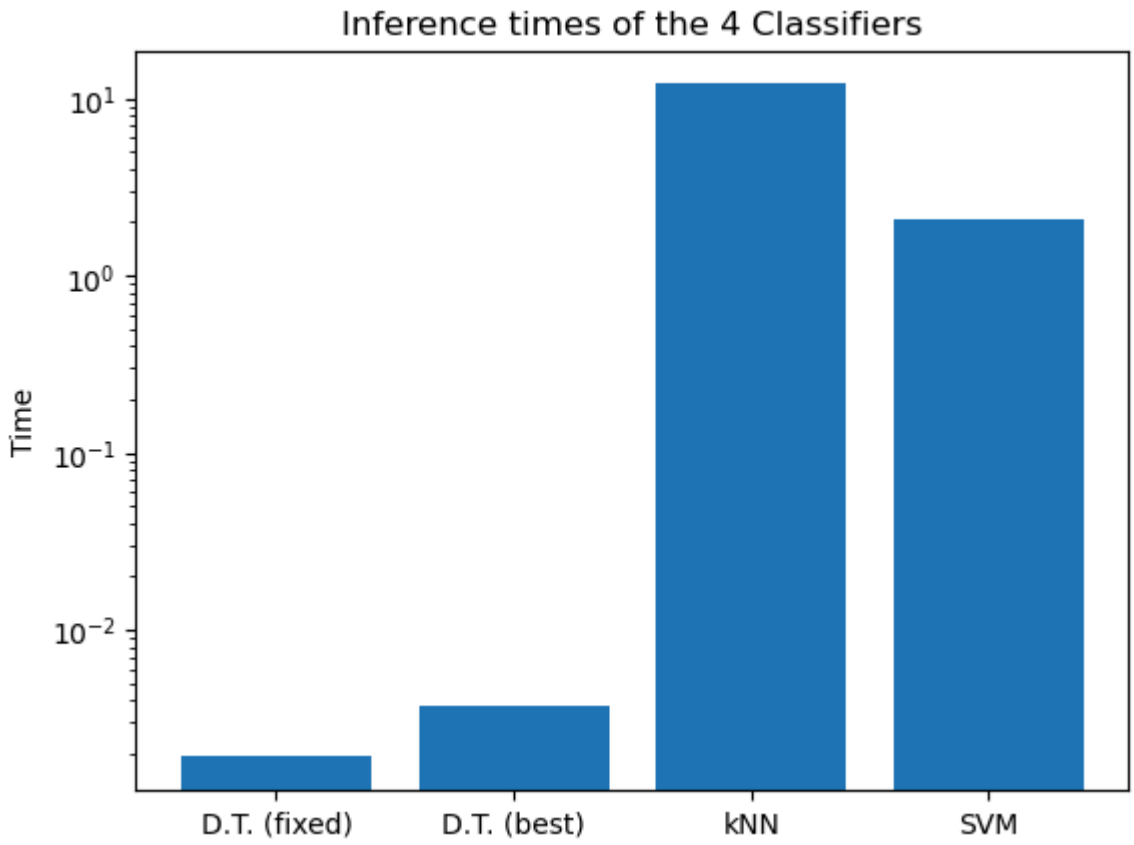
    Parameters
    ----------
    times : list
        List of 4 inference times from the previous exercises.
            - dec_tree_fixed : float
                Inference time of the decision tree with fixed parameters
```

```
            - dec_tree_bets : float
                Inference time of the decision tree with best parameters.
            - kNN : float
                Inference time of the kNN classifier.
            - SVM : float
                Inference time of the SVM classifier.
    """
    # YOUR CODE HERE
    plt.bar(['D.T. (fixed)', 'D.T. (best)', 'kNN', 'SVM'], times)
    plt.yscale('log')
    plt.ylabel('Time')
    plt.title('Inference times of the 4 Classifiers')
    plt.show()
```

In [36]:
```
# Nothing to do here, just run the cell.
accuracies = [acc_fixed, acc_best, knn_acc, svm_acc]
times = [time_fixed, time_best, knn_time, svm_time]

plot_accuracies(accuracies)
plot_times(times)
```





Question 4.3 (3 Points):

Are the following observations correct?

According to the plots above:

a4_) The decision tree model is faster in terms of inference time and yields a better accuracy than kNN and SVM.
b4_) SVM performs significantly better than kNN.
c4_) On such a large dataset, the SVM with kernel has higher inference time than the decision tree, as expected.

To answer the question, assign `True` or `False` boolean values to variables in the next cell. For example, if you think that **a4_)** is correct, define a variable `a4_` and set it to `True`, the same applies to **b4_)** and the other options. A non-correctly answered question as well as no answer (i.e. answer "None") yields 0 points for a specific question.

In [37]:
```
# YOUR CODE HERE
a4_ = False
b4_ = False
c4_ = True
```

In [38]:
```
# DO NOT DELETE THIS CELL!
assert a4_ is not None, "Store True/False!"
assert a4_ in [True, False], "Invalid Answer!"
```

In [39]:
```
# DO NOT DELETE THIS CELL!
assert b4_ is not None, "Store True/False!"
```

```
assert b4_ in [True, False], "Invalid Answer!"
```

In [40]:
```
# DO NOT DELETE THIS CELL!
assert c4_ is not None, "Store True/False!"
assert c4_ in [True, False], "Invalid Answer!"
```

Task 5: Preparation towards ensembles of trees

In the upcoming lectures, you will discuss ensemble methods for trees that aggregate and/or average single tree models to achieve better performances and/or faster runtimes compared to the ones we used here. Random Forest is a famous example where we average over trees such that the overall variance (of the average) is reduced. We will now formalize the situation:

Let's say you have $X_1, \ldots, X_B$ identically distributed random variables which are NOT necessarily independent. Let us denote the variance of a single variable X_i by σ^2 and the correlation coefficient between two X_i and X_j for $j \neq i$ by $\rho = \frac{E(X_i X_j) - E(X_i)E(X_j)}{\sigma^2}$ (keep in mind that all X_i's are identically distributed!).

In **Calculation 5.1**, show that

$$\mathrm{Var}\left(\frac{1}{B}\sum_{i=1}^{B} X_i\right) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2. \quad (1)$$

This gives some intuition about how to control the overall variance of averages. One can reduce it e.g. by a small correlation coefficient ρ and a large number of models B.

1. Apply the definition of the variance to the given average.
2. Split up the resulting double sum in parts with equal and unequal indices.
3. Apply the definition for ρ and replace the sums with the number of occurences of the respective term (how often the indices appear).
4. Rewrite this to get the desired solution.

Note: Denote in your calculation where you tackle each of this points.

Calculation 5.1 (15 Points):

$$Var(\overline{X}) = Var\left(\frac{1}{B}\sum_{i=1}^{B} X_i\right)$$

Variance of a constant times random variable is: $\frac{1}{B^2} Var\left(X_1 + X_2 + \ldots + X_B\right)$

(1)

We can write the variance as $\mathrm{Var} = \mathrm{Cov}\left(\sum_{i=1}^{B} X_i, \sum_{j=1}^{B} X_j\right)$ which gives us:

$$\sum_{i=1}^{B}\sum_{j=1}^{B}\mathrm{Cov}(X_i, X_j) =$$

$$\sum_{i=1}^{B}\mathrm{Var}(X_i) + 2\sum_{i<j}\mathrm{Cov}(X_i, X_j) =$$

---

(2)

Because the random variables are i.d., we get:

$$\mathrm{Var}(X_i) = \sigma^2 \quad \text{and} \quad \mathrm{Cov}(X_i, X_j) = \rho\sigma^2 \quad \text{for} \quad i \neq j$$

---

(3)

Rewriting the variance we get: (for i, j there are $\frac{B(B-1)}{2}$ terms)

$$\mathrm{Var}\left(\sum_{i=1}^{B} X_i\right) = B\sigma^2 + 2\sum_{i<j}\rho\sigma^2 =$$

$$B\sigma^2 + 2\frac{B(B-1)}{2}\rho\sigma^2 = B\sigma^2 + B(B-1)\rho\sigma^2$$

---

(4)

The final form of the variance:

$$Var(\overline{X}) = \frac{1}{B^2}\left(B\sigma^2 + B(B-1)\rho\sigma^2\right)$$

Distributing the terms we get:

$$Var(\overline{X}) = \frac{1}{B^2}\left(B\sigma^2 + B^2\rho\sigma^2 - B\rho\sigma^2\right)$$

$$Var(\overline{X}) = \frac{\sigma^2}{B^2}\Big(B + B^2\rho - B\rho\Big)$$

$$Var(\overline{X}) = \frac{\sigma^2}{B}\Big(1 + B\rho - \rho\Big)$$

$$Var(\overline{X}) = \sigma^2\frac{(1 - \rho + B\rho)}{B}$$

$$Var(\overline{X}) = \frac{\sigma^2 - \rho\sigma^2 + B\rho\sigma^2}{B}$$

$$Var(\overline{X}) = \frac{\sigma^2(1-\rho)}{B} + \rho\sigma^2$$